DEPI · SYSTEM ANALYSIS & DESIGN · DELIVERABLE 3 OF 6

Data Flow & System Behavior Modeling

Section 4.3 — Data flow diagrams, sequence, activity, state and class diagrams

MediCare — Clinic Management & Appointment System

ASP.NET Core MVC · N-Tier · Microsoft Azure

Team: Mohamed Munir (Team Lead) · Malak Hany · Ashraf Ibrahim · Leqaa Ibrahim · Natalie Bola

November 2026

7. Data Flow Diagrams (DFD)

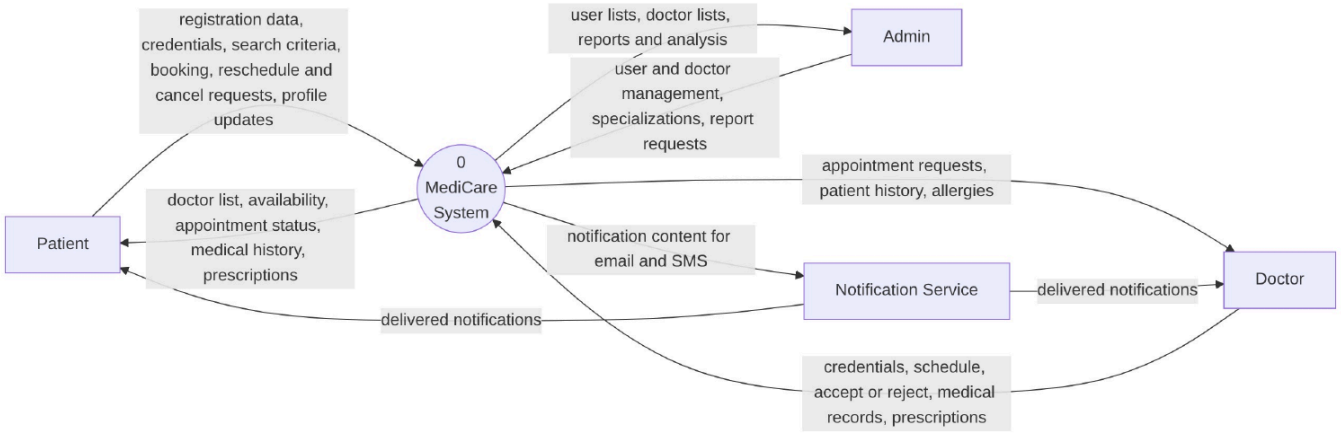
Notation: rectangle = external entity, circle = process, cylinder = data store, arrow = data flow. Items marked *(Optional)* come from the Activity Diagram.

Level 0 – Context diagram

MediCare – Data Flow Diagrams (DFD)

Aligned with the team's own deliverables: **Use Case Diagram**, **ERD**, **Software Architecture** (components and N-Tier layers), **State Diagram**, **Activity Diagram** and the **Requirements Traceability Matrix (RTM)**. Notation: **rectangle** = external entity, **circle** = process, **cylinder** = data store, **arrow** = data flow. Items marked **(Optional)** appear in the Activity Diagram but are flagged "Confirm scope" in the RTM (FR-18 Waitlist, FR-19 Allergy/Medication alerts).

1. Level 0 – Context Diagram



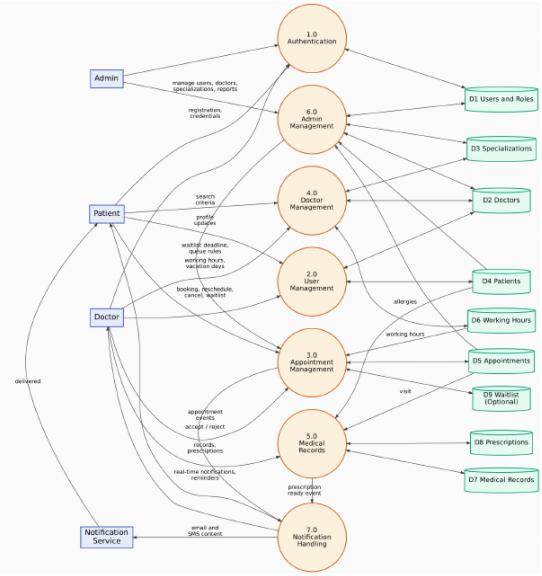
External entity	Source in the team's documents
Patient, Doctor, Admin	Use Case Diagram – primary actors
Notification Service	Use Case Diagram – supporting actor (implemented with SendGrid email and Twilio SMS per the Architecture document)

External entity	Source
Patient, Doctor, Admin	Use Case Diagram – primary actors
Notification Service	Use Case Diagram – supporting actor (implemented with SendGrid email and Twilio SMS)

Level 1 – Main processes

2. Level 1 – Main Processes

Processes follow the **component names in the Software Architecture diagram**: Authentication, User Management, Appointment Management, Doctor Management, Medical Records, Admin Management, plus Notification handling.



Process	Use cases covered	RTM
1.0 Authentication	Register, Login	FR-01, FR-02, FR-03
2.0 User Management	Manage Profile	FR-04
3.0 Appointment Management	Book, Reschedule, Cancel, Manage Appointments, View Appointments (Doctor), Waitlist (Optional)	FR-07 to FR-11, FR-18
4.0 Doctor Management	Search Doctors, View Doctor Availability, Manage Schedule	FR-05, FR-06
5.0 Medical Records	View Medical History, View Prescriptions, Create Prescription, Manage Medical Records	FR-12, FR-13, FR-14, FR-19
6.0 Admin Management	Manage Users, Manage Doctors, Manage Specializations, View Reports and Analysis	FR-15, FR-16
7.0 Notification Handling	Send Notifications	FR-17

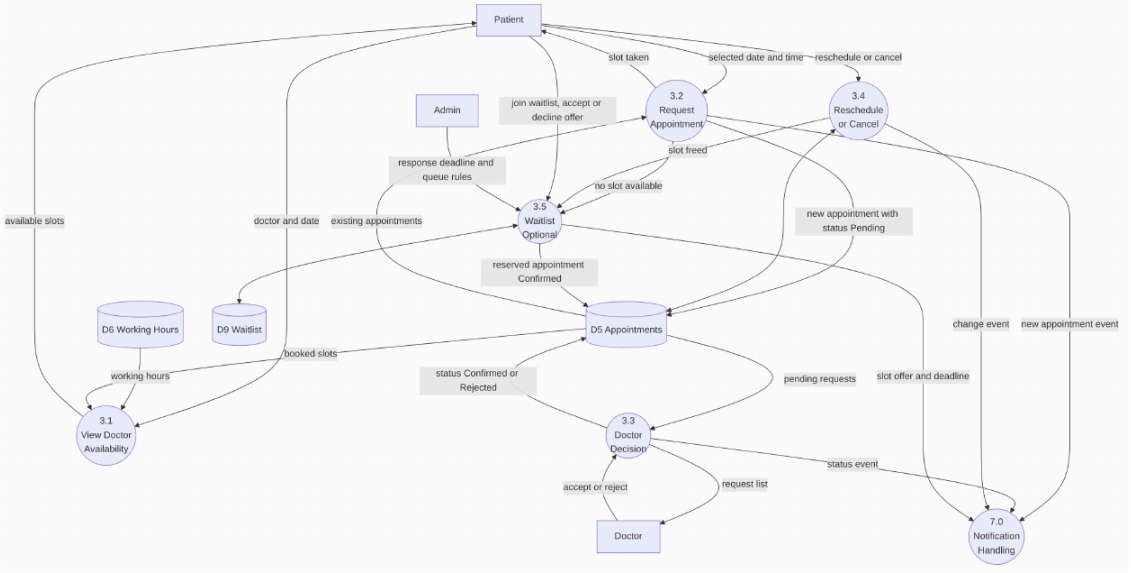
Process	Use cases covered	RTM
1.0 Authentication	Register, Login	FR-01, FR-02, FR-03
2.0 User Management	Manage Profile	FR-04
3.0 Appointment Management	Book, Reschedule, Cancel, Manage Appointments, View Appointments (Doctor), Waitlist (Optional)	FR-07 to FR-11, FR-18
4.0 Doctor Management	Search Doctors, View Doctor Availability, Manage Schedule	FR-05, FR-06
5.0 Medical Records	View Medical History, View Prescriptions, Create Prescription, Manage Medical Records	FR-12, FR-13, FR-14, FR-19
6.0 Admin Management	Manage Users, Manage Doctors, Manage Specializations, View Reports and Analysis	FR-15, FR-16
7.0 Notification Handling	Send Notifications	FR-17

Level 2 – Process 3.0 Appointment Management

Follows the State Diagram (Pending, Confirmed, Rejected, Cancelled) and the Activity Diagram (approval flow and waitlist). Balanced with Level 1.

3. Level 2 – Process 3.0 Appointment Management

Follows the State Diagram (Pending, Confirmed, Rejected, Cancelled) and the Activity Diagram (approval flow and waitlist).



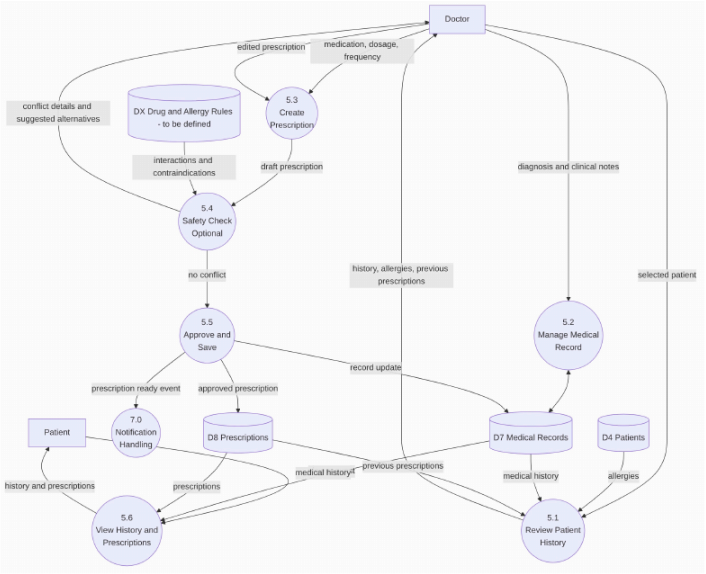
Balancing check: the flows crossing the boundary of 3.0 here match Level 1 (requests from Patient and Doctor, status back to both, reads and writes on D5 and D9, working hours from D6, events to 7.0, rules from 6.0).

Level 2 – Process 5.0 Medical Records (with prescription safety check)

Follows the Prescription Safety Workflow in the Activity Diagram.

4. Level 2 – Process 5.0 Medical Records (with prescription safety check)

Follows the Prescription Safety Workflow in the Activity Diagram: review history and allergies, create the prescription, check interactions, show conflict and alternatives, doctor edits, then approve and save.



Data stores and the ERD

Store	Source
D1 Users and Roles	ASP.NET Identity tables
D2 Doctors	ERD – Doctor
D3 Specializations	ERD multi-valued attribute; separate entity in Architecture doc
D4 Patients	ERD – Patient (allergies multi-valued)

Store	Source
D5 Appointments	ERD – Appointment
D6 Working Hours	ERD – Working hours
D7 Medical Records	ERD – Medical record
D8 Prescriptions	Architecture doc – Prescriptions with PrescriptionItems
D9 Waitlist	Activity Diagram only (Optional)
DX Drug and Allergy Rules	Activity Diagram "Check Interactions and Contraindications" (source not defined)

8. Sequence Diagrams

8.0 Booking and doctor approval (end-to-end)

Lifelines: Patient, Web Interface, Backend Controller, Business Logic, Repository / Data Access, Database, Doctor, Notification Service.

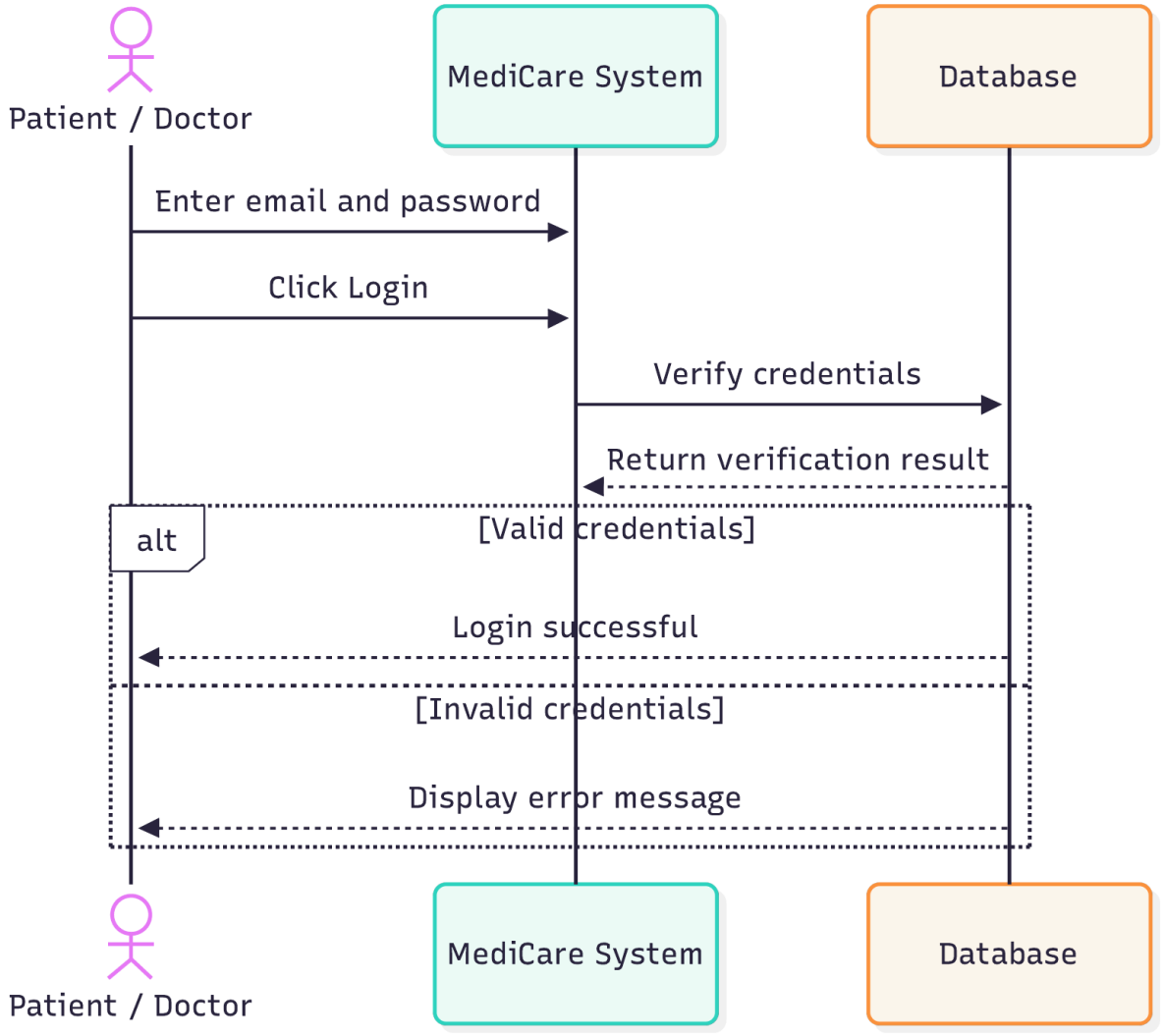


1. **Log in:** credentials → controller → business logic authenticates against the Database (verify credentials and role) → login result.
2. **Search:** patient searches → controller → business logic "find matching doctors" → repository queries doctors and schedules → results shown with slots.

3. **Book:** patient selects doctor/date/time and books → controller → business logic validates → repository checks for conflicting appointments → slot status returned.
4. **Create Pending appointment** → saved in the Database → "Booking successful"; Business logic notifies the doctor through the Notification Service (new appointment notification) → patient sees "Booking pending confirmation / Display booking status".
5. `alt` **Doctor confirms or rejects:** decision → controller → business logic → repository saves Confirmed or Rejected status → business logic notifies the patient through the Notification Service. *Doctor accepts:* updated appointment returned and displayed. *Doctor rejects:* "Reject booking request" and the patient is asked to choose another slot.

8.1 Login

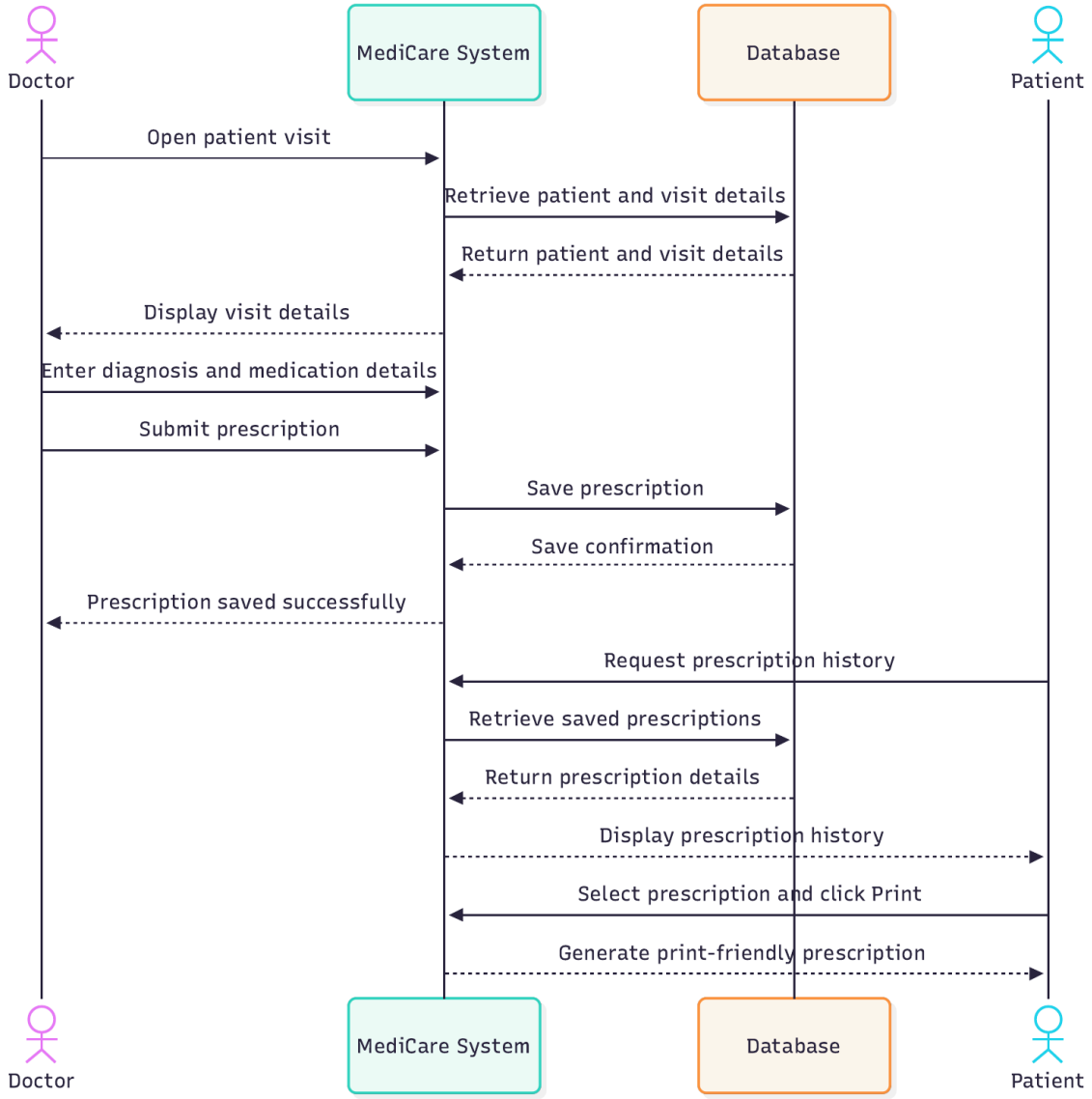
Participants: Patient / Doctor, MediCare System, Database.



1. User enters email and password and clicks Login.
2. System verifies credentials against the Database; result returned.
3. `alt` Valid credentials → "Login successful". Invalid → "Display error message".

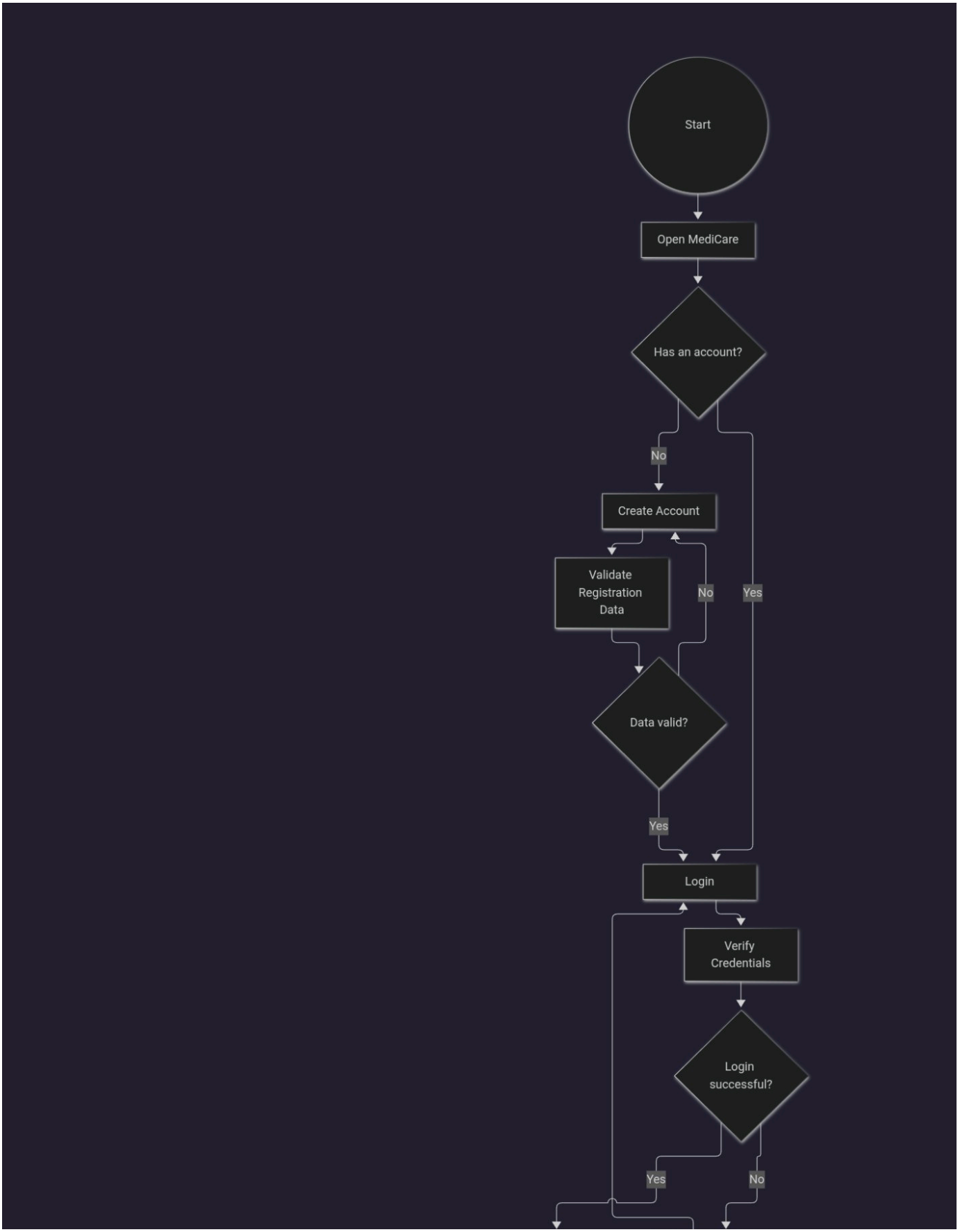
8.2 Prescription

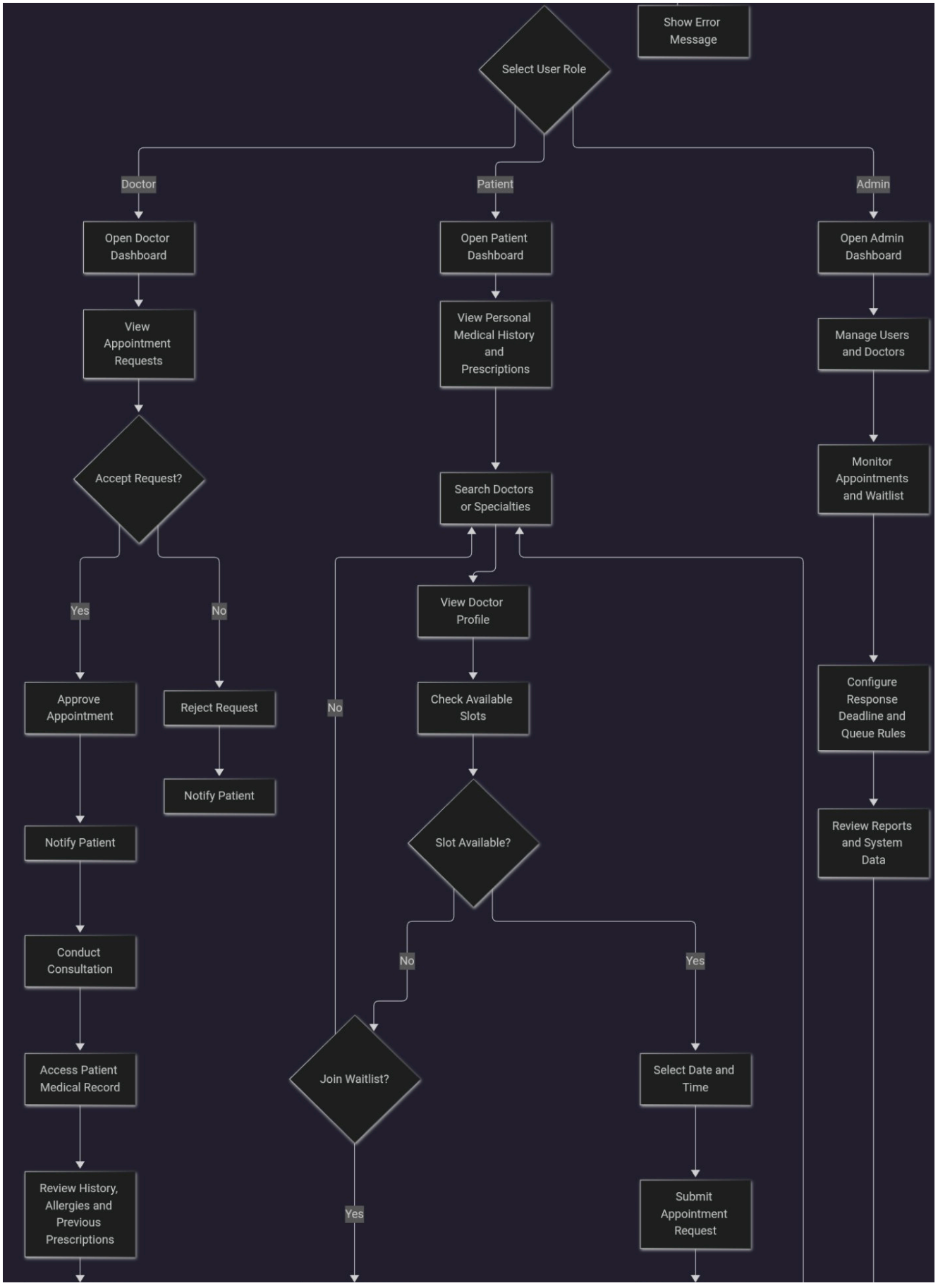
Participants: Doctor, MediCare System, Database, Patient.

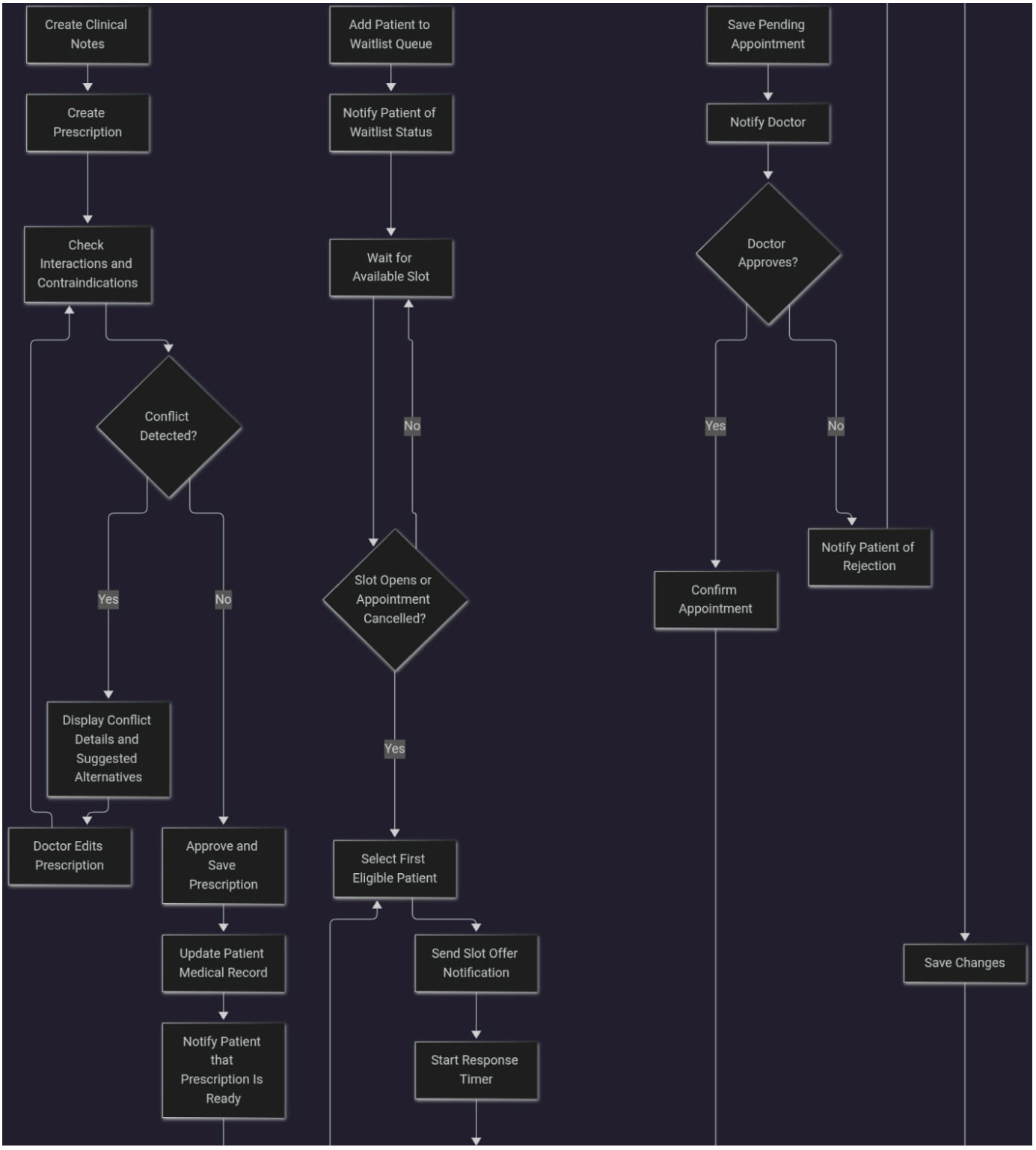


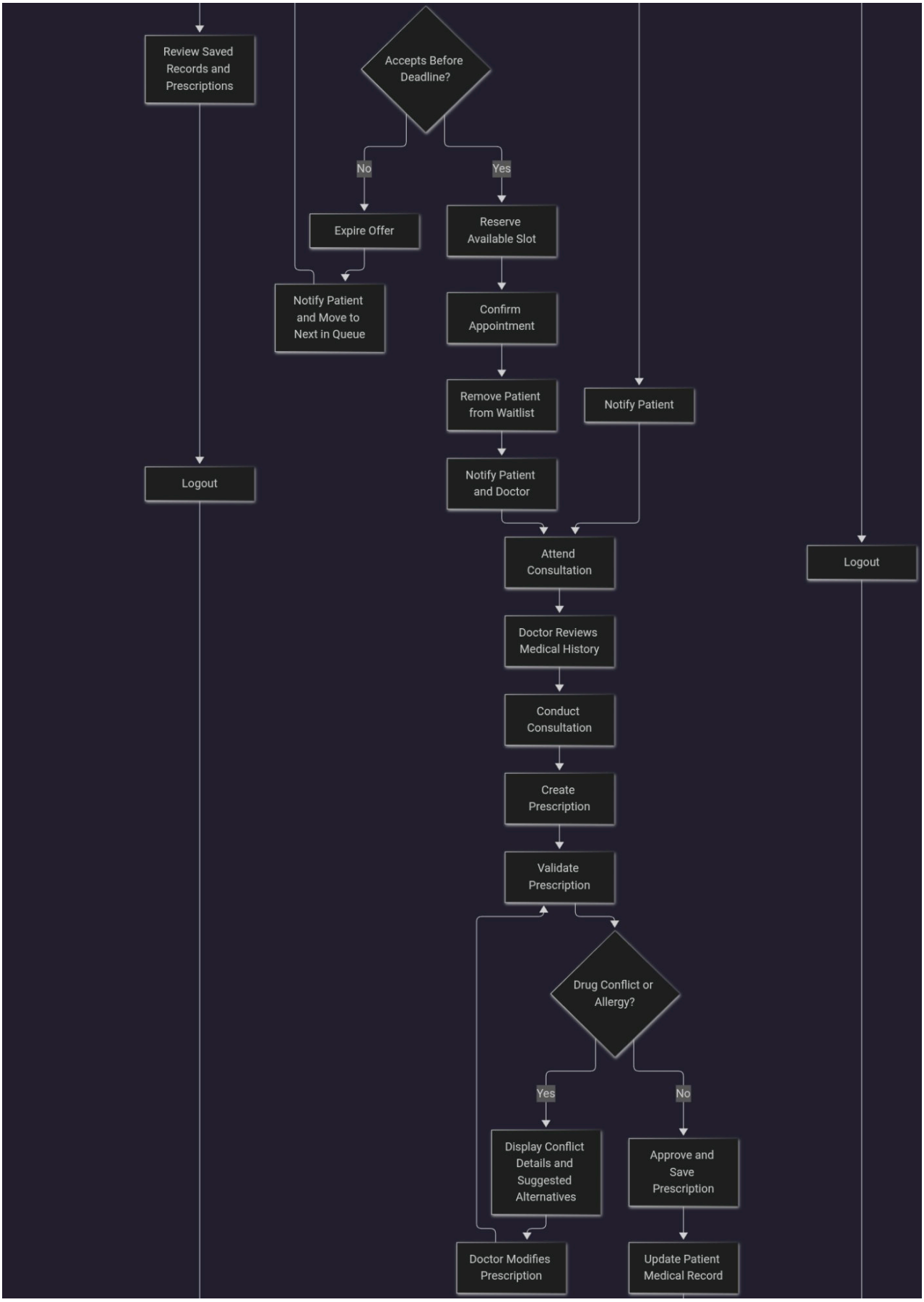
1. Doctor opens a patient visit → system retrieves patient and visit details from the Database → displayed.
2. Doctor enters diagnosis and medication details and submits the prescription → system saves it → confirmation → "Prescription saved successfully".
3. Patient requests prescription history → system retrieves saved prescriptions → displayed.
4. Patient selects a prescription and clicks Print → system generates a print-friendly prescription.

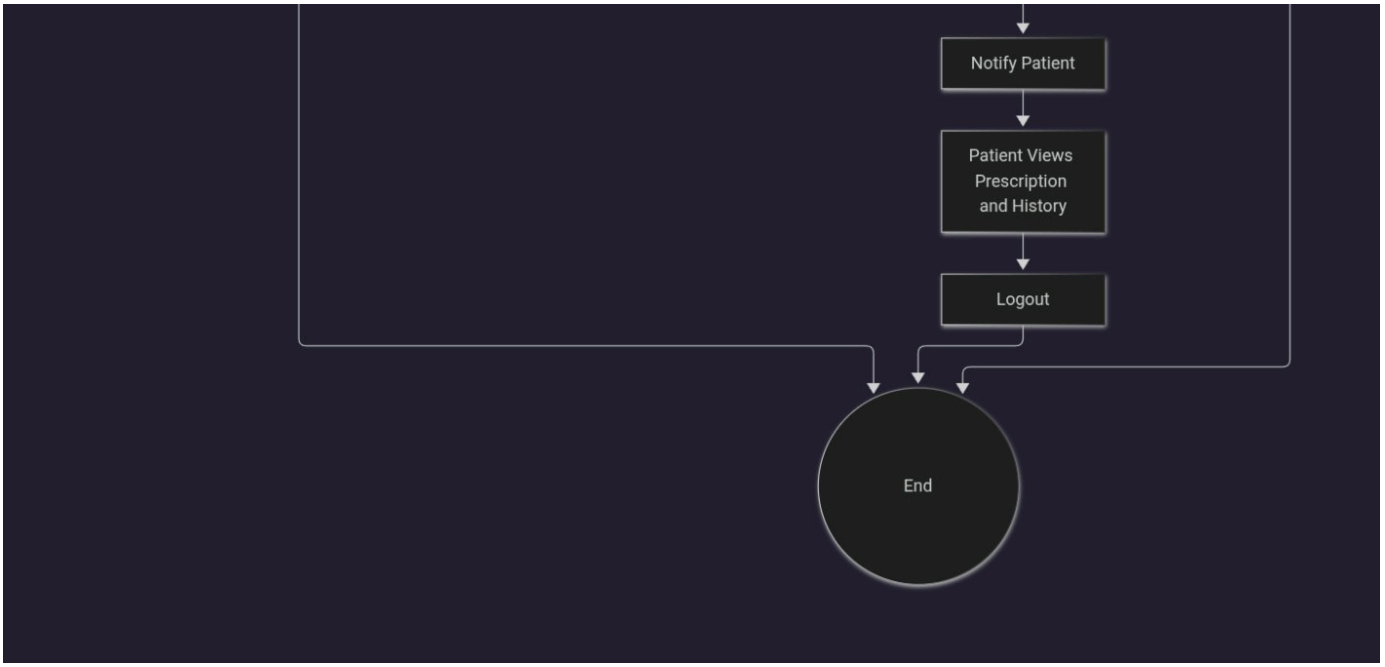
9. Activity Diagram




Show Error Message
Select User Role
Doctor
Patient
Admin
Open Doctor Dashboard
Open Patient Dashboard
Open Admin Dashboard
View Appointment Requests
View Personal Medical History and Prescriptions
Manage Users and Doctors
Accept Request?
Search Doctors or Specialties
Monitor Appointments and Waitlist
Yes
No
View Doctor Profile
Approve Appointment
Reject Request
No
Check Available Slots
Configure Response Deadline and Queue Rules
Notify Patient
Notify Patient
Slot Available?
Review Reports and System Data
Conduct Consultation
No
Yes
Access Patient Medical Record
Join Waitlist?
Select Date and Time
Review History, Allergies and Previous Prescriptions
Yes
Submit Appointment Request


Create Clinical Notes
Create Prescription
Check Interactions and Contraindications
Conflict Detected?
Yes
No
Display Conflict Details and Suggested Alternatives
Doctor Edits Prescription
Approve and Save Prescription
Update Patient Medical Record
Notify Patient that Prescription Is Ready
Add Patient to Waitlist Queue
Notify Patient of Waitlist Status
Wait for Available Slot
No
Slot Opens or Appointment Cancelled?
Yes
Select First Eligible Patient
Send Slot Offer Notification
Start Response Timer
Save Pending Appointment
Notify Doctor
Doctor Approves?
Yes
No
Confirm Appointment
Notify Patient of Rejection
Save Changes

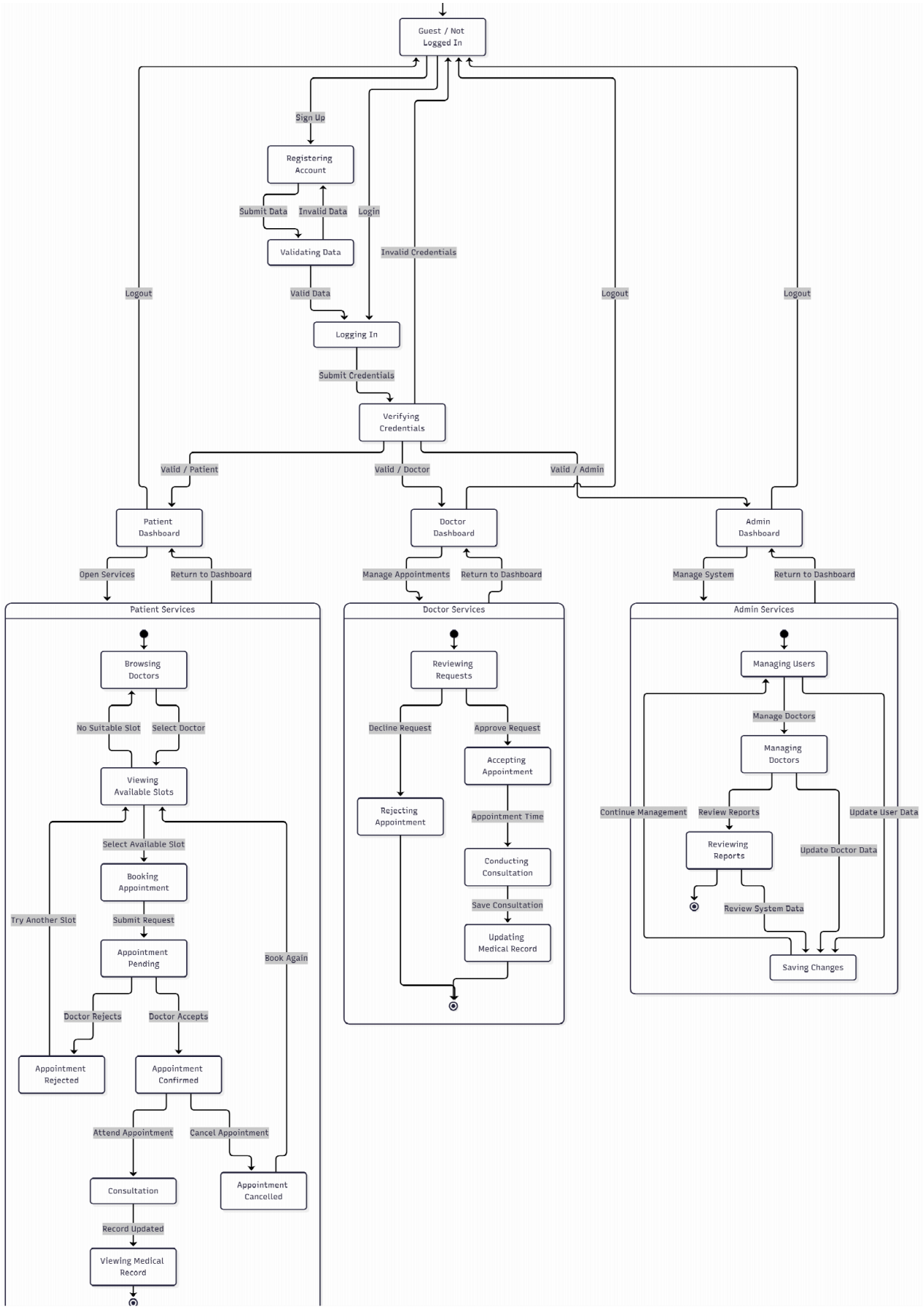

Review Saved Records and Prescriptions
Accepts Before Deadline?
No
Yes
Expire Offer
Reserve Available Slot
Notify Patient and Move to Next in Queue
Confirm Appointment
Remove Patient from Waitlist
Notify Patient
Logout
Notify Patient and Doctor
Attend Consultation
Logout
Doctor Reviews Medical History
Conduct Consultation
Create Prescription
Validate Prescription
Drug Conflict or Allergy?
Yes
No
Display Conflict Details and Suggested Alternatives
Approve and Save Prescription
Doctor Modifies Prescription
Update Patient Medical Record



Flow summary

- **Start → Open MediCare → Has an account?** No: create account → validate registration data (loop until valid). Yes: login → verify credentials (error message on failure) → **select user role**.
- **Patient:** dashboard → view medical history and prescriptions → search doctors/specialties → doctor profile → check available slots.
  - Slot available: select date/time → submit request → save pending appointment → notify doctor → *doctor approves?* Yes: confirm appointment + notify patient; No: notify patient of rejection.
  - No slot: join waitlist? → add to waitlist queue and notify status → wait for slot → when a slot opens or an appointment is cancelled: select first eligible patient → send slot offer → start response timer → accepts before deadline? Yes: reserve slot, confirm, remove from waitlist, notify patient and doctor. No: offer expires, notify and move to next in queue.
  - Then: attend consultation → doctor reviews history, conducts consultation, creates prescription → validate prescription → drug conflict/allergy? Yes: show conflict details and suggested alternatives, doctor modifies. No: approve and save, update medical record, notify patient → patient views prescription and history → logout.
- **Doctor:** dashboard → view appointment requests → accept? Approve (notify, conduct consultation, access record, review history/allergies/previous prescriptions, create clinical notes, create prescription, check interactions and contraindications → conflict? show details + alternatives, doctor edits → else approve & save, update record, notify patient) or reject (notify patient) → logout.
- **Admin:** dashboard → manage users and doctors → monitor appointments and waitlist → configure response deadline and queue rules → review reports and system data → save changes → logout.

10. State Diagram



Session and authentication states

From	Event	To
Guest / Not Logged In	Sign Up	Registering Account
Registering Account	Submit Data	Validating Data
Validating Data	Invalid Data	Registering Account
Validating Data	Valid Data	Logging In
Guest / Not Logged In	Login	Logging In
Logging In	Submit Credentials	Verifying Credentials
Verifying Credentials	Invalid Credentials	Guest / Not Logged In
Verifying Credentials	Valid / Patient · Valid / Doctor · Valid / Admin	Patient Dashboard · Doctor Dashboard · Admin Dashboard
Each dashboard	Logout	Guest / Not Logged In

Patient Services (composite state)

Browsing Doctors → *Select Doctor* → Viewing Available Slots (*No Suitable Slot* → back to Browsing Doctors) → *Select Available Slot* → Booking Appointment → *Submit Request* → **Appointment Pending**

- *Doctor Rejects* → **Appointment Rejected** → *Try Another Slot* → Viewing Available Slots
- *Doctor Accepts* → **Appointment Confirmed**
  - *Attend Appointment* → Consultation → *Record Updated* → Viewing Medical Record → end
  - *Cancel Appointment* → **Appointment Cancelled** → *Book Again* → Viewing Available Slots

Doctor Services (composite state)

Reviewing Requests →

- *Decline Request* → Rejecting Appointment → end
- *Approve Request* → Accepting Appointment → *Appointment Time* → Conducting Consultation → *Save Consultation* → Updating Medical Record → end

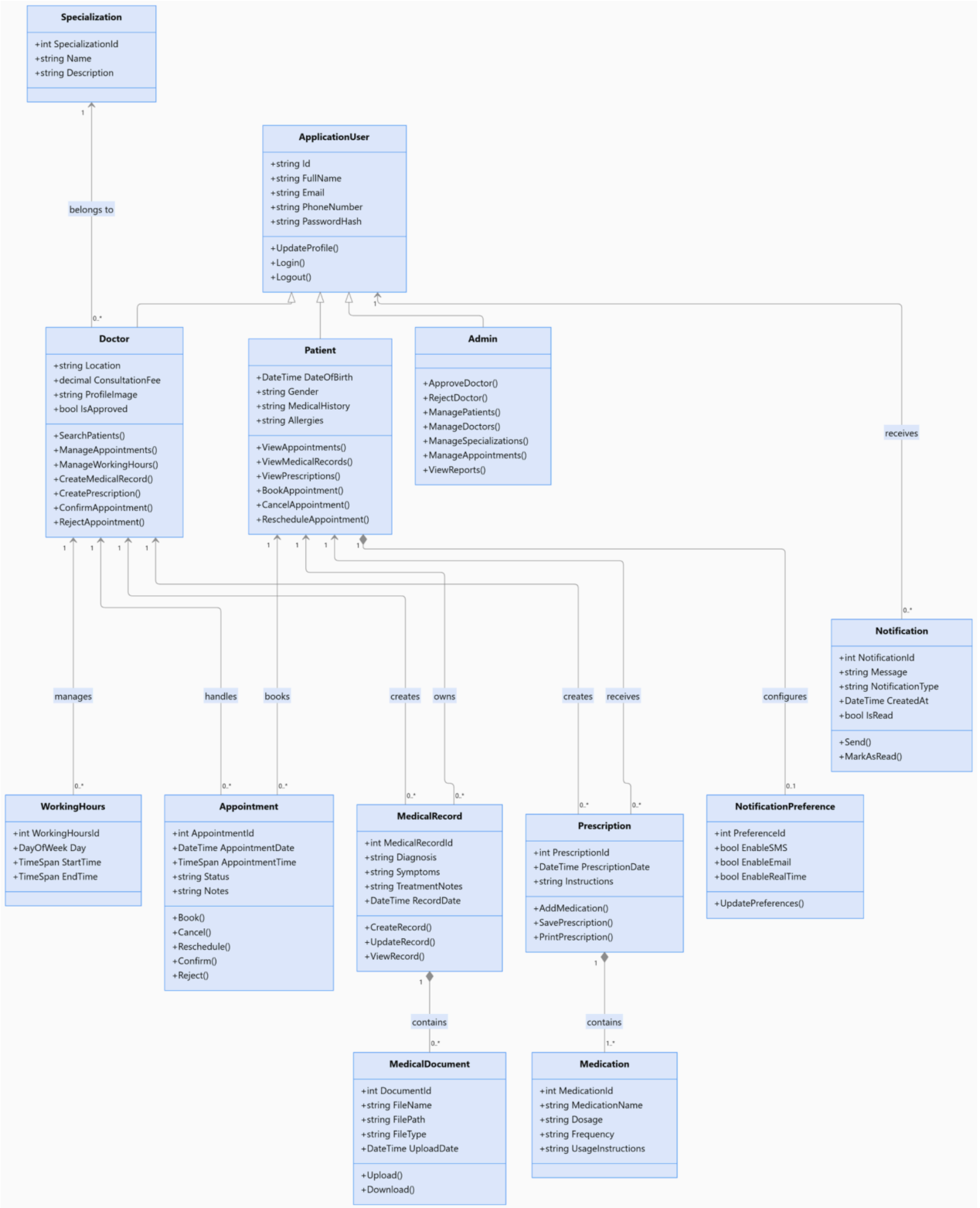
Admin Services (composite state)

Managing Users → *Manage Doctors* → Managing Doctors → *Review Reports* → Reviewing Reports (→ end), or *Update Doctor Data* / *Update User Data* / *Review System Data* → Saving Changes → *Continue Management* → Managing Users.

Appointment states in this diagram

Pending, Confirmed, Rejected, Cancelled (+ a Consultation step).

11. Class Diagram



Classes

Class	Attributes	Methods
ApplicationUser (base)	Id, FullName, Email, PhoneNumber, PasswordHash	UpdateProfile(), Login(), Logout()
Doctor (extends ApplicationUser)	Location, ConsultationFee (decimal), ProfileImage, IsApproved	SearchPatients(), ManageAppointments(), ManageWorkingHours(), CreateMedicalRecord(), CreatePrescription(), ConfirmAppointment(), RejectAppointment()
Patient (extends ApplicationUser)	DateOfBirth, Gender, MedicalHistory, Allergies	ViewAppointments(), ViewMedicalRecords(), ViewPrescriptions(), BookAppointment(), CancelAppointment(), RescheduleAppointment()
Admin (extends ApplicationUser)	—	ApproveDoctor(), RejectDoctor(), ManagePatients(), ManageDoctors(), ManageSpecializations(), ManageAppointments(), ViewReports()
Specialization	SpecializationId, Name, Description	—
WorkingHours	WorkingHoursId, DayOfWeek, StartTime, EndTime	—
Appointment	AppointmentId, AppointmentDate, AppointmentTime, Status, Notes	Book(), Cancel(), Reschedule(), Confirm(), Reject()
MedicalRecord	MedicalRecordId, Diagnosis, Symptoms, TreatmentNotes, RecordDate	CreateRecord(), UpdateRecord(), ViewRecord()
MedicalDocument	DocumentId, FileName, FilePath, FileType, UploadDate	Upload(), Download()
Prescription	PrescriptionId, PrescriptionDate, Instructions	AddMedication(), SavePrescription(), PrintPrescription()
Medication	MedicationId, MedicationName, Dosage, Frequency, UsageInstructions	—
Notification	NotificationId, Message, NotificationType, CreatedAt, IsRead	Send(), MarkAsRead()
NotificationPreference	PreferenceId, EnableSMS, EnableEmail, EnableRealTime	UpdatePreferences()

Relationships (as labelled)

- Doctor/Patient/Admin → inherit from ApplicationUser.
- Specialization — *belongs to* — Doctor (1 : 0..*).
- Doctor *manages* WorkingHours (1 : 0..*); Doctor *handles* Appointment; Patient *books* Appointment.
- Doctor *creates* MedicalRecord; Patient *owns* MedicalRecord; MedicalRecord *contains* MedicalDocument (1 : 0..*).
- Doctor *creates* Prescription; Patient *receives* Prescription; Prescription *contains* Medication (1 : 1..*).
- Patient *configures* NotificationPreference; ApplicationUser *receives* Notification (0..*).